demo

October 21, 2026

1 Module 3 Demo:

Configuring a Bidirectional LSTM CRF Model

Here, we show you how to configure the object that you will use to create the bidirectional LSTM Module.

Firstly, we have some starting variables to utilize for initialization.

[ ]:
```
import torch.nn as nn
from torchcrf import CRF

PAD_TOKEN = "<PAD>"
vocab = {PAD_TOKEN: 0, "apple" : 1, "banana" : 2, "cherry": 3}
labels = {"meat" : 1, "fruit" : 2, "unedible": 3}
embedding_dimension = 200
pad_index = vocab[PAD_TOKEN]
hidden_dimension = 256
```

We initialize the class with nn.Module, which is the base class for all neural networks in Pytorch. This means our BiLSTMCRF model will behave as a PyTorch model.

We call the base class' constructor with .init()

Then, we create an embedding layer with the required arguments to turn the token indices into dense vector representations, known as embeddings. First, we pass in the size of the vocabulary we are working with. `Embedding_dimension` is the size of the embedding vectors. Here, the dimension is 200, meaning that each word will be mapped to a 200 dimension vector. `Padding_idx` tells us which token to ignore in training.

Next, we create the LSTM layer. Again, we specify the embedding dimension, which is 200. This is the input size of the LSTM layer. The size of the LSTM's hidden state is also defined and assigned to `hidden_dimension` which in this case we will set as 256. Next, we set batch_first to true, which ensures that the input tensor has a shape compatible with input data formats in PyTorch. Next, we set `bidirectional` to true, which means that the LSTM processes the inputs both forwards and backwards.

Next, we specify a Linear Layer assigned to `hidden2label`. This maps hidden states from the LSTM to the label space. The input size of this layer is twice the hidden_dimension because the LSTM is bidirectional, so the forward and backward hidden states are concatenated. The output

size is equal to the number of labels in the dataset (in this case, 3). Each token will be assigned probabilities for each label.

Finally, we specify the CRF layer. This layer applies a Conditional Random Field on top of the LSTM outputs. The number of labels is passed as the size of the CRF layer, and batch_first=True ensures the CRF expects input data in the proper shape for the model.

[ ]:
```
class BiLSTMCRF(nn.Module):
    def __init__():
        super(BiLSTMCRF, self).__init__()
        self.embedding = nn.Embedding(len(vocab), embedding_dimension,␣
↪padding_idx=pad_index)
        self.lstm = nn.LSTM(embedding_dimension, hidden_dimension,␣
↪batch_first=True, bidirectional=True)
        self.hidden2label = nn.Linear(hidden_dimension * 2, len(labels))
        self.crf = CRF(len(labels), batch_first=True)
```